

Spark Streaming receives live input data streams and divides the data into batches, which are then processed by the Spark engine to generate the final stream of results in batches.

## Spark SQL

Spark SQL is a module for structured data processing. It supports executing SQL queries on a variety of data sources such as relational databases and hive, through a DataFrame interface. A DataFrame is a data set organised into named columns. It is conceptually equivalent to a table in a relational database. Unlike the basic Spark RDD API, the interfaces of Spark SQL provide more information about the structure of both the data and the computation being performed. Internally, Spark SQL uses this extra information to perform extra optimisations. There are several ways to interact with Spark SQL including SQL and the data set API. When computing a result, the same execution engine is used, independent of which API/language you are using to express the computation. This unification means that developers can easily switch back and forth between different APIs, based on which one provides the most natural way to express a given transformation.

## Machine Learning Library (MLib)

MLib is Spark's machine learning (ML) library. Its goal is to make practical ML scalable and easy. MLib provides tools and utilities to load ML algorithms such as classification, regression, clustering and collaborative filtering on large volumes of data sets. It supports operations such as feature extraction, transformation, dimensionality reduction and selection. It has pipelining tools for constructing, evaluating and tuning ML pipelines. We can save and load algorithms, models and pipelines. Spark MLib has utilities to work with linear algebra, statistics, data handling, etc.

## GraphX

GraphX is a component in Spark for graphs and graph-parallel computation. At a high level, GraphX extends the Spark RDD by introducing a new Graph abstraction—a directed multi-graph with properties attached to each vertex and edge. To support graph computation, GraphX exposes a set of fundamental operators (e.g., subgraph, joinVertices, and aggregateMessages) as well as an optimised variant of the Pregel API. In addition, GraphX includes a growing collection of graph algorithms and builders to simplify graph analytics tasks.

## Developing Spark applications

Let us look at how to develop a streaming application using the Apache Spark streaming module. Before that, follow the steps given below:

1. Create an application in Twitter.
2. Generate an access token and access token secret.
3. Note down the customer key and customer secret.
4. Ensure the ZooKeeper server and the Kafka broker server are up and running. Create a topic in Kafka to store the

streaming data from the following Twitter source:

```
kafka-topics --create --zookeeper zookeeperserver_host:2181
--replication-factor 1 --partitions 1 --topic rk_hadoop
```

5. Ensure Python packages such as python-kafka, python-twitter and tweepy are installed in your machine.
6. Update *KafkaTwitterStream.py* and *WordCount.py* based on the environment. Execute a Python program which consumes streaming data from Twitter and store it in *Kafka topic*. *KafkaTwitterStream.py* is configured to get streaming data for 60 seconds and the *WordCount.py* application is configured to process data with a 10-second streaming interval.
7. Open a Kafka consumer console in a new tab to ensure streaming data is stored in the Kafka topic:

```
kafka-console-consumer --zookeeper zookeeperserver_host:2181
--topic kafka_topic
```

8. Ensure HDFS and YARN service components are working fine in your Cloudera cluster. Submit the application to Spark on the YARN cluster by defining the class path, application jar path and the Spark Master as YARN, and deploy the mode as *client/cluster*. The number of drivers, executor cores and the amount of driver and executor memory is set by default.

```
spark2-submit-master yarn-deploy-mode client -conf "spark.
dynamicAllocation.enabled=false" -jars /opt/cloudera/
parcels/SPARK2/lib/spark2/examples/jars/spark-examples_2.11-
2.2.0.cloudera2.jar test1.py zookeeperserver_host:2181 kafka_
topic
```

I have disabled dynamic memory allocation, which means that executors are removed when idle. Dynamic allocation is not effective in Spark Streaming, in which data comes in every batch and executors run whenever data is available. If the executor idle timeout is less than the batch duration, executors are constantly being added and removed. However, if the executor idle timeout is greater than the batch duration, executors are never removed.

9. We can process the streaming data using Spark until the data stream from Twitter is ingested by Kafka.  
Here is the code for *KafkaTwitterStream.py*:

```
#!/usr/bin/python
from tweepy.streaming import StreamListener
from tweepy import OAuthHandler
from tweepy import Stream
from kafka import SimpleProducer, KafkaClient
import time

access_token = "access_token"
access_token_secret = "access_token_secret"
consumer_key = "consumer_key"
```